

Conditional Rendering in React (In-Depth)

Conditional rendering allows React components to render different outputs based on state or props.

Step 1 Understand JavaScript Conditions in JSX

- React allows JavaScript conditions inside the JSX return block.
- Basic way: Use `if` statements outside JSX to decide what to render.

```
function Greeting(props) {  
  if (props.isLoggedIn) {  
    return <h1>Welcome back!</h1>;  
  }  
  return <h1>Please log in.</h1>;  
}
```

- This method works well if you have a few mutually exclusive UI blocks.

Step 2 Inline Conditional Rendering Using Ternary Operator

- To embed conditions directly in JSX, use the ternary operator `condition ? true : false`.

```
<div>  
  {isLoggedIn ? <Dashboard /> : <LoginPage />}  
</div>
```

- Ideal for quick, simple branches within JSX.

Step 3 Conditional Rendering with Logical `&&`

- Render an element only if a condition is true:

```
{hasNotifications && <NotificationIcon />}
```

- If `hasNotifications` is `false`, React renders nothing.

Step 4 Return `null` to Render Nothing

- Sometimes you want to hide a component. Return `null` in the render:

```
function Warning({ show }) {  
  if (!show) return null;  
  return <div>Warning!</div>;  
}
```

- Useful for toggling visibility.

Step 5 Multiple Conditions with `switch` or Conditional Functions

- For many conditions, use a `switch` statement or helper function:

```
function renderContent(page) {  
  switch(page) {  
    case 'home': return <Home />;  
    case 'about': return <About />;  
    default: return <NotFound />;  
  }  
}  
  
function App() {  
  return <div>{renderContent(currentPage)}</div>;  
}
```

Step 6 Conditional CSS Class or Style Changes

- You can toggle CSS classes based on conditions:

```
<div className={isError ? 'error' : 'info'}>  
  {message}  
</div>
```

- Or use inline styles conditionally:

```
<button style={{ backgroundColor: isActive ? 'blue' : 'gray' }}>Click me</button>
```

Rendering Lists in React Step-by-Step)

Lists are used to render multiple components dynamically.

Step 1 Using `map()` to Render Arrays

- The most common way to render lists is the JavaScript array `.map()` function.

```
const fruits = ['Apple', 'Banana', 'Cherry'];  
<ul>
```

```
{fruits.map(fruit => <li key={fruit}>{fruit}</li>)}  
</ul>
```

Step 2 Use Unique `key` Props

- Always supply a unique `key` prop for each list item to help React track item changes.
- Use stable unique identifiers — do NOT use array index if the array could reorder.

Step 3 Render Lists of Objects

- Use `.map()` on objects. Especially common with API data.

```
const users = [{id: 1, name: 'Alice'}, {id: 2, name: 'Bob'}];  
<ul>  
  {users.map(user => <li key={user.id}>{user.name}</li>)}  
</ul>
```

Step 4 Extract List Items into Subcomponents

- For clarity, make list items a separate component:

```
function User({ user }) {  
  return <li>{user.name}</li>;  
}  
  
function UserList({ users }) {  
  return <ul>{users.map(user => <User key={user.id} user={user} />)}</ul>;  
}
```

Step 5 Lists with Fragments for Multiple Elements per Item

- If each list item returns multiple elements, use React Fragments:

```
{items.map(item => (  
  <React.Fragment key={item.id}>  
    <dt>{item.term}</dt>  
    <dd>{item.description}</dd>  
  </React.Fragment>  
))}
```

Step 6 Update List Dynamically with State

- Use React state to add, remove, or update items dynamically.

```
const [items, setItems] = React.useState(initialItems);  
  
function addItem(newItem) {  
  setItems([...items, newItem]);  
}  
  
function removeItem(id) {
```

```
setItems(items.filter(item => item.id !== id));  
}
```

Styling in React Detailed Guide)

React supports multiple approaches for styling components, ranging from quick inline styles to advanced CSS-in-JS.

Step 1 Inline Styles

- Apply styles directly to elements with JavaScript objects:

```
<div style={{ color: 'red', fontWeight: 'bold' }}>Important</div>
```

- Styles must be camelCase (backgroundColor instead of background-color).
- Great for dynamic one-off styles but can become verbose.

Step 2 External CSS Stylesheets

- Create .css files and import them into your component or app.

```
/* styles.css */  
.title {  
  color: blue;  
  font-size: 24px;  
}
```

```
import './styles.css';  
<h1 className="title">Hello</h1>
```

- This keeps styles separate and caching efficient.

Step 3 CSS Modules for Scoped Styles

- Rename your style files to Component.module.css.
- Import as an object and apply styles via object keys:

```
/* Button.module.css */  
.btn {  
  background-color: green;  
}
```

```
import styles from './Button.module.css';  
<button className={styles.btn}>Click</button>
```

- Prevents global scope pollution by scoping CSS locally.

Step 4 Styled-Components CSS in JS

- Install styled-components library.

```
import styled from 'styled-components';

const Button = styled.button`
  background: palevioletred;
  color: white;
  padding: 10px 20px;
  border-radius: 4px;
`;

<Button>Styled Button</Button>
```

- Supports props-based dynamic styling:

```
const Button = styled.button`
  background: ${props => props.primary ? 'blue' : 'gray'};
  color: white;
`;

<Button primary>Primary</Button>
```

- Ideal for component-oriented, theme-based styling.

Step 5 Conditional Styling

- Toggle classes or styles based on props or state:

```
<div className={isSelected ? 'selected' : ''}>Item</div>

<div style={{ opacity: isDisabled ? 0.5 : 1 }}>Content</div>
```

- Combine with CSS modules or styled-components for best practices.

Step 6 Advanced Topics

- Use CSS variables and media queries for responsiveness.
- Use libraries like `clsx` or `classnames` for conditionally combining CSS classes.
- Ensure accessibility by using semantic HTML and sufficient contrast.

This detailed step-wise guide provides students with not only the practical code but also explanations of when and why to use each method for conditional rendering, lists, and styling in React. This will build a strong foundation to write clean, maintainable React code with advanced patterns and real-world readiness. [1](#) [2](#) [3](#) [4](#)

If you want, the next step can be providing mini practice exercises or sample projects to solidify these concepts.

<https://legacy.reactjs.org/docs/conditional-rendering.html>

<https://react.dev/learn/conditional-rendering>

<https://blog.logrocket.com/react-conditional-rendering-9-methods/>

<https://kinsta.com/blog/react-conditional-render/>

https://www.w3schools.com/react/react_conditional_rendering.asp

<https://www.freecodecamp.org/news/react-conditional-rendering/>

<https://www.geeksforgeeks.org/reactjs/reactjs-conditional-rendering/>

https://www.youtube.com/watch?v=E_C39RK79zs

[https://www.reddit.com/r/reactjs/comments/ov2azx/whats the best practice for conditional rendering/](https://www.reddit.com/r/reactjs/comments/ov2azx/whats_the_best_practice_for_conditional_rendering/)